Master Project: Bridging Vision and Language—A Multi-Image AI Pipeline for Context-Aware NPC Storytelling

Hassan Coulibaly

April 15, 2026

Abstract

Modern interactive media frequently struggles with ludonarrative dissonance, where dynamically driven player actions conflict with static, pre-scripted game narratives. While recent advancements in Vision-Language Models (VLMs) have enabled narrative extraction from single, static images, these approaches lack the temporal and sequential context required to interpret complex gameplay. This project introduces a novel, multi-modal machine learning pipeline designed to generate context-aware, dynamic dialogue for Non-Playable Characters (NPCs) based on sequential gameplay screenshots. The proposed architecture separates visual perception from narrative generation to strictly ground the AI in gameplay reality. First, a VLM sequentially processes in-game images to extract key entities, actions, and environmental states, structuring this data into a chronological scene graph. Second, a Large Language Model (LLM) utilizes this structured data—constrained by established interactive storytelling principles—to generate dramatically meaningful and credible NPC dialogue. By filtering raw visual data into a strict JSON framework before text generation, the pipeline reduces LLM hallucination and arbitrary content creation. Evaluation covers all three pipeline stages: per-frame VLM extraction is assessed via F1 scoring against ground-truth binary labels; final dialogue quality is assessed via an automated LLM-as-judge method scoring Grounding, Coherence, and Engagement on a 1–5 scale; and a human rating study template is provided for manual validation. Across three evaluated runs on *Genshin Impact* gameplay footage, the pipeline achieved average scores of 4.67/5 for Grounding, 4.67/5 for Coherence, and 5.00/5 for Engagement, with zero hallucinated entities detected. This approach demonstrates a scalable method for bridging computer vision and natural language processing, allowing games to dynamically translate visual player events into coherent, immersive lore without sacrificing narrative integrity.

1 Introduction

The landscape of interactive media and video games has evolved from simple, linear progression into expansive, open-world environments where player agency dictates the flow of events. As rendering technologies and physics engines have advanced to create highly realistic simulations, the systems governing in-game storytelling have often lagged behind. In most contemporary open-world games, Non-Playable Character (NPC) dialogue remains heavily pre-scripted. While branching dialogue trees offer the illusion of choice, they are ultimately finite and struggle to account for the emergent, unpredictable actions players take in a physics-driven world.

This master's project implements a prototype *talking agent* pipeline: a batch of time-ordered gameplay screenshots is summarized per frame by a VLM, merged into a single structured *scene graph*, and passed to an LLM that produces short NPC dialogue under explicit grounding constraints. The implementation uses a configurable Google Gemini model (default `gemini-2.5-flash`;

overridable via `GEMINI_MODEL` in `.env` or the `--model` flag) for both the vision and language calls, with extraction forced to `application/json` for machine-parseable state.

A key design goal is evaluation completeness. Prior work on VLM pipelines often evaluates only the perception stage. This project contributes a two-track evaluation: automated F1 for Stage 1, and an LLM-as-judge scoring method for Stage 3 that measures Grounding, Coherence, and Engagement while also detecting hallucinated entities.

2 Background

A prototype system for AI & Interactive Storytelling in Cultural Heritage was developed for the Dunhuang Murals (specifically the Jataka stories) to move beyond passive viewing [?]. The system includes mural segmentation to explain individual elements, interactive knowledge graphs that trigger cinematic visual effects when clicked, and a creative reconstruction tool where users build their own comic-style narratives. An AI agent (built on the COZE platform and GPT models) acts as an approachable guide, providing real-time explanations and creative suggestions while drawing from a curated knowledge base to prevent "hallucinations". User testing showed high scores in attractiveness, novelty, and stimulation, suggesting the system helps users form deeper emotional and intellectual connections with heritage.

3 System Architecture and Implementation

3.1 Design goals

The design follows three principles aligned with the research abstract: (1) **temporal grounding**—images are processed in a user-defined order (e.g., lexicographic filename order as a proxy for capture order); (2) **structured mediation**—raw pixels are never passed directly to the dialogue model; only JSON derived from vision passes through; (3) **constrained narration**—the final prompt instructs the model to avoid inventing entities, locations, or plot beats absent from the scene graph.

3.2 Stage 1: Per-frame visual extraction

For each frame index $i = 0, 1, \ldots$, the pipeline sends the screenshot and a fixed instruction prompt to the VLM. The model must return a JSON object with fields including `location`, `time_of_day`, `player_status`, `recent_action`, `notable_enemies_or_objects`, and `visible_entities` (a list of short strings). Extraction uses a low temperature (0.2) to favor factual stability. Each result is wrapped with metadata: `frame_index`, `source_image`, and `observation`.

3.3 Stage 2: Consolidation into a chronological scene graph

Per-frame observations are merged into one JSON *scene graph* suitable for dialogue generation. Two modes are implemented:

- **LLM consolidation (default).** A second call to the same model merges the ordered frame list into a graph with `timeline_summary`, `environment` (`location`, `time_of_day`, `conditions`), `entities` (with `id`, `name_or_description`, `type`, `first_frame`, `last_frame`), `events` (`frame_index`, `description`, `involved_entity_ids`), and `player_arc` (`status_notes`, `notable_actions`). The consolidator is instructed not to invent content absent from the observations.

- **Simple stitch (`--simple-consolidate`).** A deterministic merge (no extra API call) concatenates actions, collects entity strings, and builds a minimal graph; useful for cost-sensitive runs or ablations.

3.4 Stage 3: NPC dialogue generation

The LLM receives only the consolidated scene graph and a block of *storytelling constraints*: ground claims in the JSON, avoid filling gaps with specific unsupported lore, keep a short campfire recap in second person, and avoid omniscient narration. Dialogue generation uses a higher temperature (0.7) for phrasing variety while the input remains fixed structured state.

4 Codebase, Data Layout, and Reproducibility

The repository is organized as follows (paths relative to project root):

```
talking_agent/
  src/
    main_pipeline.py          <- full 3-stage pipeline
    evaluator.py              <- Stage 1 F1 scoring
    dialogue_evaluator.py     <- Stage 3 LLM-as-judge scoring
    eda_visualize.py          <- EDA plots and figures
  data/
    test_screenshots/         <- preferred input folder
    extracted_state/          <- *_frames.json, *_scene_graph.json
    generated_stories/        <- *_npc_dialogue.txt
    analysis/figures/         <- PNG exports from eda_visualize.py
  test_images/                <- legacy samples (fallback)
  evaluation/
    ground_truth_labels.csv   <- binary frame labels for Stage 1
    dialogue_scores.csv       <- LLM-as-judge results
    dialogue_scores.json      <- same data, structured JSON
    human_scoring_template.csv
  .env.example
  requirements.txt
```

4.1 Execution

From the repository root, after `pip install -r requirements.txt` and setting `GEMINI_API_KEY` in a local `.env` file (see `.env.example`):

```
python src/main_pipeline.py
python src/main_pipeline.py --images-dir path/to/screenshots
python src/main_pipeline.py --simple-consolidate
python src/main_pipeline.py --model gemini-2.5-flash \
    --max-frames 15 --max-image-side 1024
```

By default, the script uses `data/test_screenshots` if it contains images; otherwise it falls back to `test_images`. Output files are timestamped (UTC) under `data/extracted_state/` and `data/generated_stories/`

If the VLM extraction stage completes but the LLM merge or dialogue step fails, the run can be resumed without repeating vision calls:

```
python src/main_pipeline.py \
    --from-frames-json data/extracted_state/<run_id>_frames.json
```

4.2 API keys and version control

Secrets must not be committed. Contributors should copy `.env.example` to `.env` and add the key locally. If a key was ever committed, it should be revoked in the Google AI console and replaced with a new key; historical git commits do not erase exposure.

5 Evaluation

Evaluation spans all three pipeline stages. Stage 1 is evaluated with automated binary classification metrics. Stage 3 is evaluated with an LLM-as-judge method and supported by a human rating instrument. Stage 2 (scene graph consolidation) is assessed indirectly through the Grounding dimension of the Stage 3 judge: if the consolidation omits or distorts key entities, the dialogue scores will reflect it.

5.1 Stage 1: VLM extraction accuracy (F1)

The script `src/evaluator.py` reads `evaluation/ground_truth_labels.csv` with columns `Image_ID`, `Condition_Tested`, `Ground_Truth`, and `VLM_Prediction` (binary 0/1). It reports precision, recall, and F1 per condition and a macro-averaged F1. Conditions include binary checks such as `location_detectable` and `action_described`. Rows are populated with labels from the experimenter's rubric and pipeline outputs.

5.2 Stage 3: Dialogue quality via LLM-as-judge

The script `src/dialogue_evaluator.py` implements an automated judge to evaluate the final dialogue against its source scene graph. For each run, it loads `*_scene_graph.json` and `*_npc_dialogue.txt`, constructs a structured prompt, and calls a Gemini model configured as an expert evaluator. The judge scores three dimensions on a 1–5 integer scale:

- **Grounding**: Are all concrete claims (entity names, locations, actions, outcomes) traceable to the scene graph? Penalises invented content.
- **Coherence**: Is the narrative internally consistent, logically ordered, and expressed in a single voice?
- **Engagement**: Does the output feel like a believable, immersive campfire story from an NPC companion?

The judge also returns a list of **hallucinated entities**—proper nouns present in the dialogue but absent from the scene graph. Results are saved to `evaluation/dialogue_scores.csv` and `evaluation/dialogue_scores.json`.

Table 1: LLM-as-judge dialogue evaluation results (scale: 1–5).

Run ID	Grounding	Coherence	Engagement	Hallucinations
20260408_182648	4	5	5	0
20260408_185849	5	5	5	0
20260408_193459	5	4	5	0
Average	**4.67**	**4.67**	**5.00**	**0**

Results

Table 1 reports scores for three runs on *Genshin Impact* gameplay footage, evaluated using `gemini-2.5-flash` as the judge model.

The single Grounding deduction in run `182648` was attributed to the word *flames* at the end of the dialogue—a campfire detail inferred by the model rather than present in the scene graph. The Coherence deduction in run `193459` reflects a minor event-ordering inconsistency: the Qucusaur Tyrant was mentioned after chronologically later encounters. No hallucinated proper nouns (character names, locations, item names) were detected in any run.

5.3 Human evaluation

The template `evaluation/human_scoring_template.csv` is pre-populated with the three run IDs (`20260408_182648`, `20260408_185849`, `20260408_193459`). Human raters are provided the corresponding `.txt` files from `data/generated_stories/` and asked to score each dialogue on Coherence, Grounding, and Engagement using the same 1–5 scale as the automated judge. The recommended study design uses 3–5 raters scoring all three samples, which would allow inter-rater agreement to be computed and compared against the automated scores.

6 Limitations and Future Work

Frame order is user-supplied (e.g., filename order), not inferred from video timestamps. VLM mistakes propagate into the graph and dialogue; consolidation may over- or under-merge events. The current LLM-as-judge evaluation relies on the same Gemini model family used for generation, which may introduce evaluator bias; future work could use an independent judge model. The pipeline does not yet integrate live game APIs or native scene graphs from engines; extending to engine telemetry would tighten grounding further. The human evaluation study has not yet been conducted; completing it would allow direct comparison of automated and human scores across the three evaluation dimensions.